Programação funcional

Aula 05 - Funções de alta ordem*

*Baseado no material disponível em https://haskell.pesquisa.ufabc.edu.br/haskell

Funções como tipos

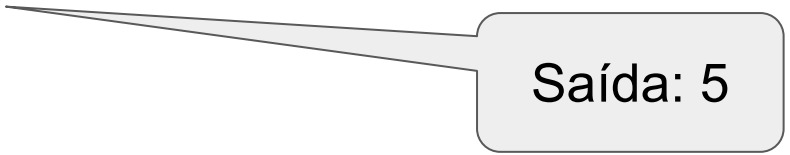
- Função: tipo de dado
  - Que pode ser usado como entrada e saída de funções
  - Ex:

```
soma :: Int -> Int -> Int soma x y = x+y
-- reescrevendo
soma :: Int -> (Int -> Int) soma x y = x+y
soma3 = soma 3
soma3 2
```

Funções como tipos

- Função: tipo de dado
  - Que pode ser usado como entrada e saída de funções
  - Ex:

```
soma :: Int -> Int -> Int soma x y = x+y
-- reescrevendo
soma :: Int -> (Int -> Int) soma x y = x+y
soma3 = soma 3
soma3 2
```



Funções como tipos

- Função: tipo de dado
  - Que pode ser usado como entrada e saída de funções
  - Ex:

```
soma :: Int -> Int -> Int soma x y = x+y
-- reescrevendo
soma :: Int -> (Int -> Int) soma x y = x+y
soma3 = soma 3
soma3 2
```

Aplicação parcial: pode ser utilizada em qualquer função de mais de 1 argumento.

Funções como argumento

- Exemplo:

```
duasVezes :: (a -> a) -> a -> a
duasVezes f x = f (f x)

duasVezes (*2) 3
duasVezes reverse [1,2,3]
```

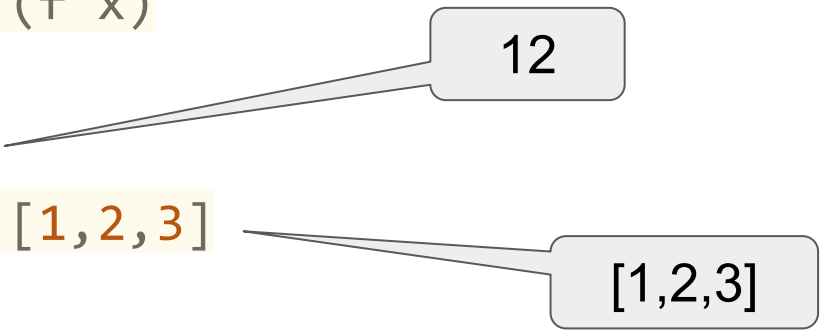
Deixou em aberto o 1o argumento da função de multiplicação

Funções como argumento

- Exemplo:

```
duasVezes :: (a -> a) -> a -> a
duasVezes f x = f (f x)

duasVezes (*2) 3
duasVezes reverse [1,2,3]
```



Funções de alta ordem

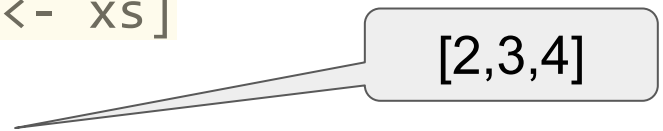
- Funções de alta ordem → funções que:
  - recebem funções como argumento ou
  - devolvem uma função

Funções de alta ordem para listas

- Considere o padrão: `[f x | x <- xs]`
  - Muito comum quando precisamos transformar os itens de uma lista
  - Equivalente com a função **map:**
    - ```
      map :: (a -> b) -> [a] -> [b]
      map f xs = [f x | x <- xs]
      ```
    - Ex.:
      - `map (+1) [1,2,3]`
      - `map even [1,2,3]`
      - `map reverse ["ola", "mundo"]`
      ```

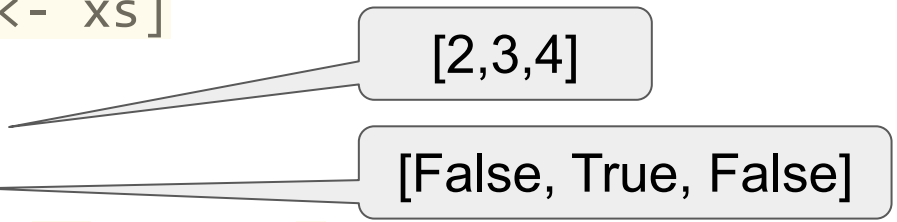
Funções de alta ordem para listas

- Considere o padrão: `[f x | x <- xs]`
  - Muito comum quando precisamos transformar os itens de uma lista
  - Equivalente com a função **map:**
    - ```
      map :: (a -> b) -> [a] -> [b]
      map f xs = [f x | x <- xs]
      ```
    - Ex.:
      - `map (+1) [1,2,3]`
      - `map even [1,2,3]`
      - `map reverse ["ola", "mundo"]`


      ```

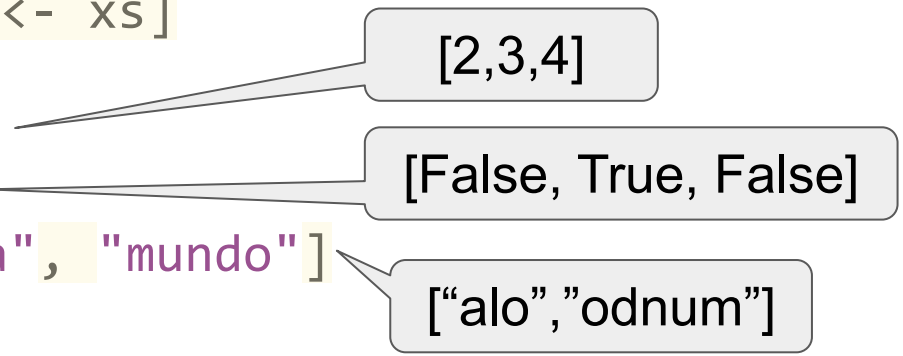
Funções de alta ordem para listas

- Considere o padrão: `[f x | x <- xs]`
  - Muito comum quando precisamos transformar os itens de uma lista
  - Equivalente com a função **map:**
    - ```
      map :: (a -> b) -> [a] -> [b]
      map f xs = [f x | x <- xs]
      ```
    - Ex.:
      - `map (+1) [1,2,3]`
      - `map even [1,2,3]`
      - `map reverse ["ola", "mundo"]`


      ```

Funções de alta ordem para listas

- Considere o padrão: `[f x | x <- xs]`
  - Muito comum quando precisamos transformar os itens de uma lista
  - Equivalente com a função **map:**
    - ```
      map :: (a -> b) -> [a] -> [b]
      map f xs = [f x | x <- xs]
      ```
    - Ex.:
      - `map (+1) [1,2,3]`
      - `map even [1,2,3]`
      - `map reverse ["ola", "mundo"]`

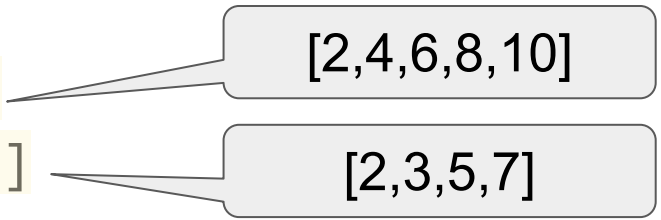

      ```

Funções de alta ordem para listas

- Definição recursiva de
  - ```
    map :: (a -> b) -> [a] -> [b]
    map f []     = []
    map f (x:xs) = f x : map f xs
    ```
    ```

Filter

- Outro padrão recorrente:
  - Filtragem de elementos
    - `[x | x <- [1..10], even x]`
      `[x | x <- [1..10], primo x]`



Filter

- Reescrevendo os exemplos anteriores
  - ```
    filter :: (a -> Bool) -> [a] -> [a]
    filter p xs = [x | x <- xs, p x]
    ```
- Devolve uma lista de todos os valores cujo o predicado **p** de **x** devolve **True**
    ```

Filter

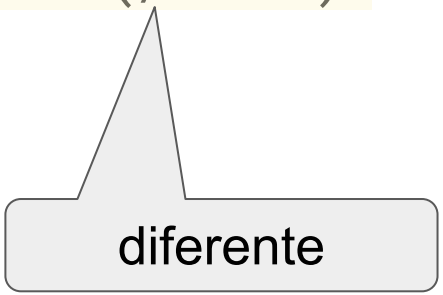
- Reescrevendo os exemplos anteriores:
  - ```
    filter even [1..10]
    filter primo [1..10]
    ```
    ```

Filter

- Reescrevendo os exemplos anteriores:
  - `filter even [1..10]`
    `filter primo [1..10]`
- Exemplo com função λ
  - `filter (\s -> length s < 4) [“ola”, “mundo”]`

Filter

- Passando funções parciais como parâmetro:
  - ```
    filter (>5) [1..10]
    filter (/= ' ') "abc def ghi"
    ```


    ```

Filter

- Passando funções parciais como parâmetro:
  - ```
    filter (>5) [1..10]
    filter (/= ' ') "abc def ghi"
    ```
    ```

Filter

- **Filter** recursivo
  - ```
    filter :: (a -> Bool) -> [a] -> [a]
    filter p []                   = []
    filter p (x:xs) | p x         = x : filter p xs
                    | otherwise = filter p xs
    ```
    ```

Map e filter

- **map** e **filter** costumam ser utilizadas juntas
  - ```
    somaQuadPares :: [Int] -> Int
    somaQuadPares ns = sum [n^2 | n <- ns, even n]
    ```
    ```

Map e filter

- **map** e **filter** costumam ser utilizadas juntas
  - ```
    somaQuadPares :: [Int] -> Int
    somaQuadPares ns = sum [n^2 | n <- ns, even n]
    ```
- Reescrevendo:
  - ```
    somaQuadPares :: [Int] -> Int
    somaQuadPares ns = sum (map (^2) (filter even ns))
    ```

Operador pipe

- Operador **$**:
  - Separa aplicações de funções
  - Permitindo remover parênteses

```
> :t ($)
($) :: (a -> b) -> a -> b
```

- Exemplo:
  - ```
    somaQuadPares :: [Int] -> Int
    somaQuadPares ns = sum
                     $ map (^2)
                     $ filter even ns
    ```
- Execução de baixo para cima
    ```

Funções de alta ordem

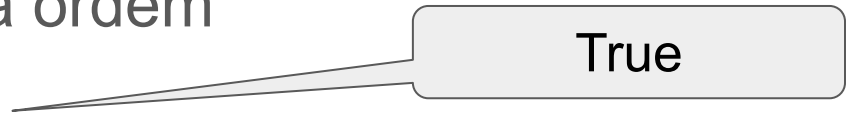
- Outras funções de alta ordem

```
all even [2,4,6,8]
any odd [2,4,6,8]
takeWhile even [2,4,6,7,8]
dropWhile even [2,4,6,7,8]
```

Funções de alta ordem

- Outras funções de alta ordem

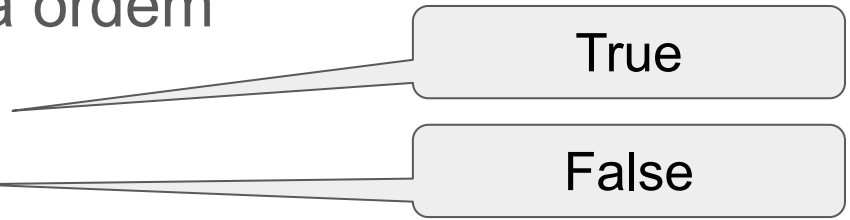
```
all even [2,4,6,8]
any odd [2,4,6,8]
takeWhile even [2,4,6,7,8]
dropWhile even [2,4,6,7,8]
```



Funções de alta ordem

- Outras funções de alta ordem

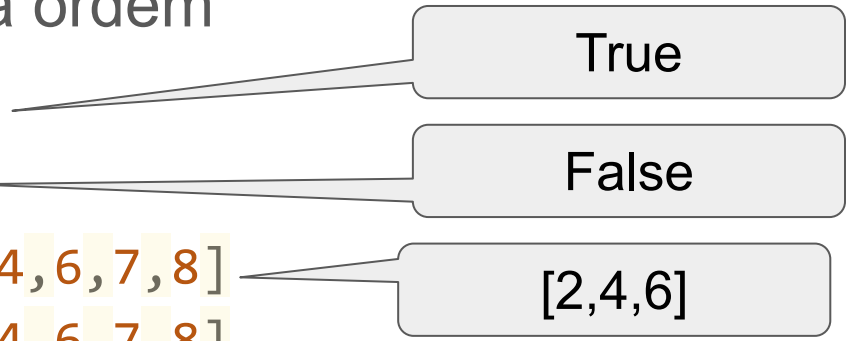
```
all even [2,4,6,8]
any odd [2,4,6,8]
takeWhile even [2,4,6,7,8]
dropWhile even [2,4,6,7,8]
```



Funções de alta ordem

- Outras funções de alta ordem

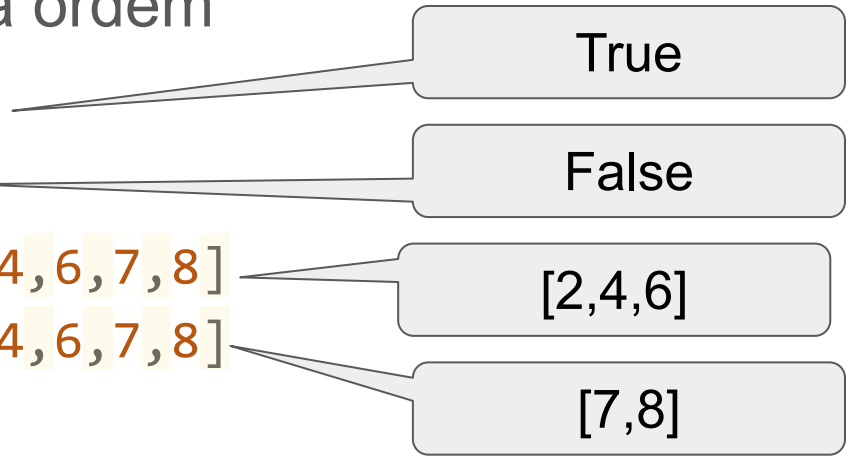
```
all even [2,4,6,8]
any odd [2,4,6,8]
takeWhile even [2,4,6,7,8]
dropWhile even [2,4,6,7,8]
```



Funções de alta ordem

- Outras funções de alta ordem

```
all even [2,4,6,8]
any odd [2,4,6,8]
takeWhile even [2,4,6,7,8]
dropWhile even [2,4,6,7,8]
```



Folding

Folding

- Recapitulando algumas funções recursivas

```
sum []     = 0
sum (x:xs) = x + sum xs

product []     = 1
product (x:xs) = x * product xs

length []     = 0
length (_:xs) = 1 + length xs
```

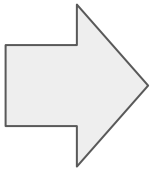
Folding

- Generalizando

```
sum []     = 0
sum (x:xs) = x + sum xs

product []     = 1
product (x:xs) = x * product xs

length []     = 0
length (_:xs) = 1 + length xs
```



```
foo [] = v
foo (x:xs) = f x (foo xs)
```

Folding

```
foo [] = v
foo (x:xs) = f x (foo xs)
```

- Essa função se chama **foldr**

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f v [] = v
foldr f v (x:xs) = f x (foldr f v xs)
```

Folding

- Essa função se chama **foldr**

```
foo [] = v
foo (x:xs) = f x (foo xs)
```




```
a1 : (a2 : (a3 : []))
```

```
a1 `f` (a2 `f` (a3 `f` v))
```

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f v [] = v
foldr f v (x:xs) = f x (foldr f v xs)
```

Ex.:

```
foldr (+) 0 [1,2,3]
```

```
1 + (2 + (3 + 0))
```

Folding

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f v [] = v
foldr f v (x:xs) = f x (foldr f v xs)
```

- Exercício:
  - Função product usando **foldr**

Folding

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f v [] = v
foldr f v (x:xs) = f x (foldr f v xs)
```

- Exercício:
  - Função product usando **foldr:**
    - `product = foldr (*) 1`

Folding

```haskell
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f v [] = v
foldr f v (x:xs) = f x (foldr f v xs)
```

- Exercício:
  - Função product usando **foldr:**
    - `product = foldr (*) 1`
  - Função **length:**

Folding

```haskell
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f v [] = v
foldr f v (x:xs) = f x (foldr f v xs)
```

- Exercício:
  - Função product usando **foldr:**
    - `product = foldr (*) 1`
  - Função **length:**
    - `length = foldr (\_ n -> n + 1) 0`

Folding caudal

- Relembrando a soma com recursão na cauda

```haskell
sum :: Num a => [a] -> a
sum ns = sum' 0 ns
  where
    sum' v []     = v
    sum' v (x:xs) = sum' (v+x) xs
```

Folding caudal

- Relembrando a soma com recursão na cauda

```haskell
sum :: Num a => [a] -> a
sum ns = sum' 0 ns
  where
    sum' v []     = v
    sum' v (x:xs) = sum' (v+x) xs
```

Isso é implementado pela função **foldl**

Folding caudal

```haskell
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f v [] = v
foldr f v (x:xs) = f x (foldr f v xs)
```

- Função **foldl**

```haskell
foldl :: (a -> b -> a) -> a -> [b] -> a
foldl f v []     = v
foldl f v (x:xs) = foldl f (f v x) xs
```

Folding caudal

```haskell
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f v [] = v
foldr f v (x:xs) = f x (foldr f v xs)
```

- Função **foldl**

```haskell
foldl :: (a -> b -> a) -> a -> [b] -> a
foldl f v []     = v
foldl f v (x:xs) = foldl f (f v x) xs
```

```haskell
1 : (2 : (3 : []))
  => (([] : 1) : 2) : 3
  => ((0 + 1) + 2) + 3
```

Folding caudal

```haskell
foldl :: (a -> b -> a) -> a -> [b] -> a
foldl f v []     = v
foldl f v (x:xs) = foldl f (f v x) xs
```

- Função **foldl**
  - Como ficariam **sum** e **product**?

```haskell
sum = foldl (+) 0

product = foldl (*) 1
```

Folding caudal

```
foldl :: (a -> b -> a) -> a -> [b] -> a
foldl f v []     = v
foldl f v (x:xs) = foldl f (f v x) xs
```

- Função **foldl**
  - E a função **length**?

```
length = foldr (\_ n -> 1+n) 0
length = foldl (??) 0
```

Folding caudal

```haskell
foldl :: (a -> b -> a) -> a -> [b] -> a
foldl f v []     = v
foldl f v (x:xs) = foldl f (f v x) xs
```

- Função **foldl**
  - E a função **length**?

```haskell
length = foldr (\_ n -> n + 1) 0
length = foldl (\n _ -> n + 1) 0
```

foldr *vs* **foldl**

- Como escolher?
  - Suponha:

```
foldl (&&) False [False, False, False, False]
foldr (&&) False [False, False, False, False]
```

  - foldl:
    - `((False && False) && False) && False ...`
  - foldr:
    - `False && (False && (False && (False ...`

foldr *vs* **foldl**

- Como escolher?
  - Suponha:

```
foldl (&&) False [False, False, False, False]
foldr (&&) False [False, False, False, False]
```

  - foldl:
    - `((False && False) && False) && False ...`
  - foldr:
    - `False && (False && (False && (False ...`

Curto-circuito

foldr *vs* **foldl**

- Alguns critérios
  - Se a lista passada como argumento pode ser infinita:
    - foldr
  - Se o operador (função) pode gerar curto circuito
    - foldr
  - Senão:
    - foldl, pois pode gerar otimizações

Composição de funções

- Na matemática:
  - ***f.g*** define uma função ***z*(*x*) = *f*(*g*(*x*))**
- Em haskell:

```
(.) :: (b -> c) -> (a -> b) -> (a -> c)
f . g = \x -> f (g x)
```

Composição de funções

- Propriedades:
  - Associativa:
    - `(f . g) . h == f . (g . h)`
  - Elemento neutro **id**:
    - `f . id = id . f = f`
- Permitem o uso do **foldr**

```haskell
-- cria uma função que é a composição de uma lista
-- de funções
compose :: [a -> a] -> (a -> a)
compose = foldr (.) id
```

Composição de funções

- Exemplos

```haskell
-- A função fn
fn x = ceiling (negate (tan (cos (max 50 x))))
-- Pode ser reescrita
fn = ceiling . negate . tan . cos . max 50
```

```haskell
somaDosQuadradosImpares :: Integer
somaDosQuadradosImpares =
  sum (takeWhile (<10000) (filter odd (map (^2) [1..])))
-- ou
somaDosQuadradosImpares =
  sum . takeWhile (<10000) . filter odd . map (^2) $ [1..]
```

Referências

- Material do prof. Emilio Francesquini
  - https://haskell.pesquisa.ufabc.edu.br/haskell